



Refactoring : Switch Statements

- Cas d'utilisation concret (NeoLoad)
- Code



Cas d'utilisation concret (NeoLoad)

NeoLoad veut accéder au Vault de ses clients.

Il y a plusieurs logiciels de Vault existant avec chacun leurs propre API -> fonction différente pour chaque Vault

Cas d'utilisation concret (NeoLoad)

```
def getVault(client):  
    if client.vault == "AWS":  
        fonction1  
    elif client.vault == "Azure":  
        fonction2  
    elif client.vault == "GCP":  
        fonction3  
    elif client.vault == "OnPrem":  
        fonction4  
    elif client.vault == "Hashicorp":  
        fonction5  
    elif client.vault == "K8S":  
        fonction6  
    elif client.vault == "Vaultless":  
        fonction7  
    elif client.vault == "Other1":  
        fonction8  
    elif client.vault == "Other2":  
        fonction9  
    elif client.vault == "Other3":  
        fonction10  
    elif client.vault == "Other4":  
        fonction11  
    elif client.vault == "Other5":  
        fonction12  
    elif client.vault == "Other6":  
        fonction13  
    elif client.vault == "Other7":  
        fonction14
```

```
def lockVault(client):  
    if client.vault == "AWS":  
        fonction1  
    elif client.vault == "Azure":  
        fonction2  
    elif client.vault == "GCP":  
        fonction3  
    elif client.vault == "OnPrem":  
        fonction4  
    elif client.vault == "Hashicorp":  
        fonction5  
    elif client.vault == "K8S":  
        fonction6  
    elif client.vault == "Vaultless":  
        fonction7  
    elif client.vault == "Other1":  
        fonction8  
    elif client.vault == "Other2":  
        fonction9  
    elif client.vault == "Other3":  
        fonction10  
    elif client.vault == "Other4":  
        fonction11  
    elif client.vault == "Other5":  
        fonction12  
    elif client.vault == "Other6":  
        fonction13  
    elif client.vault == "Other7":  
        fonction14
```

Solution : utiliser des classes, héritages et polymorphismes (interface)

```
client1 = ac.AWS()  
client2 = ac.HashiCorp()  
  
client1.getVault()  
client2.getVault()
```



Avantages/Désavantages

Avantages :

- Beaucoup plus lisible pour des switch complex/enchaînement de if
- Si on ajoute une nouvelle condition (un nouvel animal, un nouveau vault), on doit modifier chaque fonction avec switch/if

Désavantages :

- Inutile s'il y a peu de if
- Peut être en opposition avec design pattern
- N'a pas moins de ligne de code

A decorative graphic on the left side of the slide consisting of two overlapping parallelograms. The front one is blue and the back one is a light greenish-blue. They are set against a dark navy blue background with diagonal stripes.

Merci de votre
écoute

